
UNIT 12 SORTING ALGORITHMS

Structure

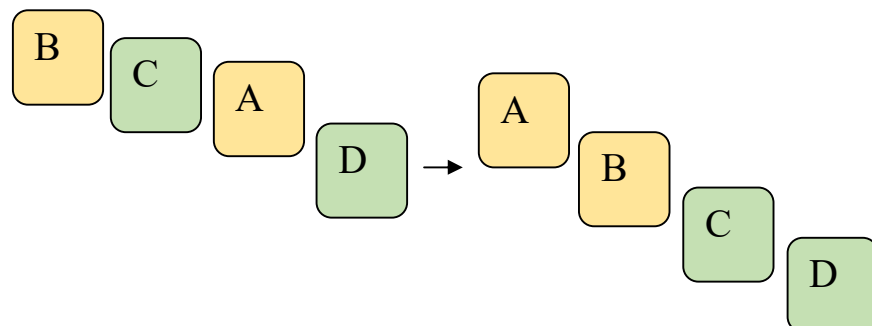
- 12.0 Introduction
- 12.1 Objectives
- 12.2 Why Study Algorithms?
- 12.3 Categories of Sorting Algorithms
- 12.4 Bubble Sort
- 12.5 Selection Sort
- 12.6 Insertion Sort
- 12.7 Merge Sort
- 12.8 Quick Sort
- 12.9 Comparing Various Sorting Algorithms
- 12.10 Python's Built-In Sorting Function
- 12.11 Summay
- 12.12 Solutions/Answers to Check Your Progress
- 12.13 Further Readings

12.0 Introduction

Sorting algorithms are directly related to databases as there is a need to sort records available in the database. Suppose, the database consists of details of students, then there is a need for names in alphabetical order, roll no in ascending/ descending order, etc. Sorting is a preliminary step involved while dealing with databases and thus is an important area to study in computer science as every project deals with a dataset. Certain algorithms have been developed for sorting the data, and this area is still an area of research.

In this chapter, we will discuss sorting algorithms, namely, merge sort, selection sort, bubble sort, etc. Each of the algorithms is demonstrated using examples and python codes. Let's study sorting algorithms one by one.

"Sorting is the method of arranging elements in either ascending or descending order."



12.1 Objectives

After going through this unit, you should be able to

- Understand the concept of sorting;
- Remember various terminologies related to sorting
- Articulate various sorting algorithms
- Implement python codes for performing sorting

12.2 Why Study Sorting Algorithms

In the field of Computer Science, everything is related to databases. When we talk about a database that contains various columns, sorting algorithms are needed to arrange the data in a particular order (increasing/ decreasing). Sorting is an important category of algorithms in the field of Computer Science and a lot of research is still going on.

12.3 Categories of Sorting Algorithms

Sorting algorithms are generally compared on the following parameters:

1. *Number of Comparisons*

The most important parameter to judge a sorting algorithm is the number of comparisons done to sort a given list of items. In this chapter, we will discuss both comparison sorting algorithms.

The comparison sorting algorithms consist of merge sort, quick sort, bubble, insertion, and selection sort.

Whereas, non-comparison sorting algorithms consist of count sort, bucket sort, and radix sort.

2. *Memory Usage*

The sorting algorithms are usually divided into two types, i.e., in-place and not in-place algorithm.

Sorting algorithm that do not need additional memory for sorting process is known as in-place algorithms.

3. *Stability*

The algorithm that preserves the relative order of same elements during sorting is known as **stable sorting algorithm**. That means, if two items have same value, the item that appeared first in the unsorted list should also appear first in the sorted list.

4. *External vs Internal Sort*

Algorithms that use main memory during sorting are called **internal sort**.

Algorithms that use external memory devices such as disk or tape during sorting are known as **external sort**.

Let's discuss various sorting algorithms as listed below one by one:

- a. Bubble Sort
- b. Selection Sort
- c. Insertion Sort
- d. Merge Sort
- e. Quick Sort

You will learn about the category of each algorithm i.e., whether an algorithm is in-place or not, stable or not, during detailed discussion of particular sorting algorithm.

Check your Progress – I

- a. What is the difference between external and internal sorting algorithms?
- b. What do you mean by term stable algorithm?
- c. Define the term in-place algorithm.

12.4 Bubble Sort

Bubble sort is the simplest algorithm that works by comparing each pair of elements and swapping elements if they are in the wrong order. The algorithm continues iteration until no more swaps are required. Bubble sort is not suitable for large datasets due to its high time complexity.

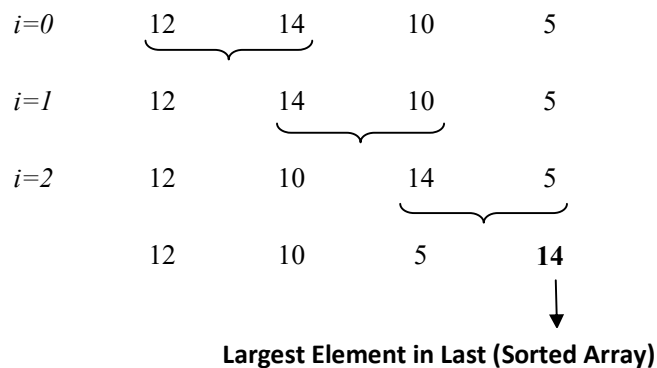
Bubble sort does not require additional memory space, i.e., in-place and is stable sorting algorithm.

Working of Bubble Sort

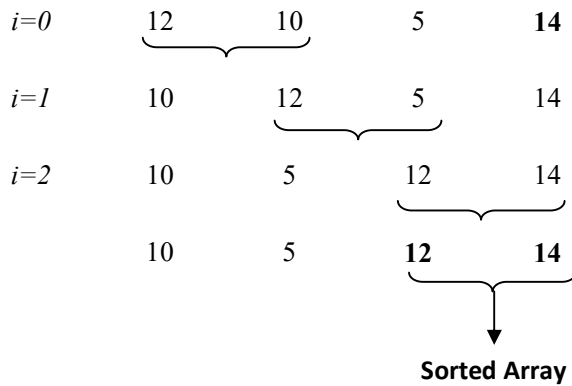
1. The array is sorted using multiple passes.
2. The largest element goes to the end of array after first pass. The second largest element goes to second last position after second pass and so on.
3. In all passes, we compare all adjacent elements and swap of larger element is before the smaller element.

Let's understand this with help of an example:

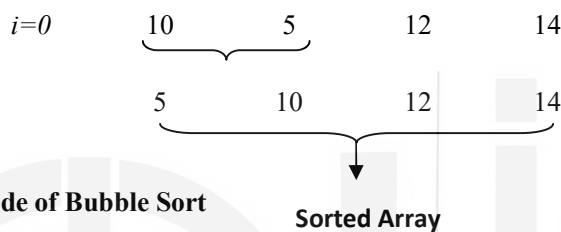
First Pass:



Second Pass:



Third Pass:



Pseudocode of Bubble Sort

```
def Bubblesort (A)
    for i in range (len(A)):
        for j in range (len(A)-1, i, -1):
            if (A[j] < A[j-1]):
                swap (A, j, j-1)
```

Performance Analysis:

	Time Complexity	Reason
Worst Case	$O(n^2)$	When array is arranged in descending order
Average Case	$O(n^2)$	Irrespective of order of elements
Best Case	$O(n^2)$	Already sorted array

Space Complexity: Bubble sort does not require additional memory space i.e., space complexity is $O(1)$.

Check Your Progress-II

- Given an array of numbers 23, 90, 21, 54, 76. Sort the array using bubble sort.
- Sort the given array of numbers using bubble sort.
67, 21, 12, 10, 9, 7, 5
- Is bubble sort a stable algorithm? Why/ Why not?

12.5 Selection Sort

Selection sort works by repeatedly selecting the smallest element from the unsorted

array and swapping it with the first unsorted element. The process continues till the array is sorted. The algorithm is easy to understand and does not require additional memory space for sorting. But it does not preserve the relative order of elements with the same value i.e., unstable sorting algorithm.

Working of Selection Sort

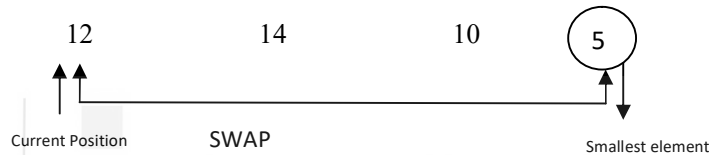
1. Find the minimum element in the unsorted array and swap it with element at current position.
2. Repeat this process until all elements are in correct order i.e., the array is sorted.

Let's understand this with the help of an example:

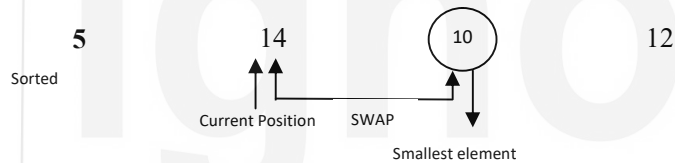
Considering the given array as:

12 14 10 5

Step 01



Step 02



Step 03



Sorted Array

5 10 12 14

Pseudocode of Selection Sort

```
def Selectionsort (A):
```

```
    for i in range (len (A)):
```

```
        min = i
```

```
        for j in range (i+1, len (A)):
```

```
            if (A[j] < A[min])
```

```
                min = j
```

```
        swap (A, min, i)
```

Performance Analysis

	Time Complexity	Reason
Worst Case	$O(n^2)$	The time complexity is irrespective of order of the elements
Average Case	$O(n^2)$	
Best Case	$O(n^2)$	

Space Complexity: It is an in-place sorting algorithm i.e., does not require additional memory space i.e., space complexity is $O(1)$.

Check Your Progress-III

- Given an array of numbers 23, 90, 21, 54, 76. Sort the array using selection sort.
- Show all steps to sort given array 90, 45, 11, 5, 67, 43 using selection sort.

12.6 INSERTION SORT

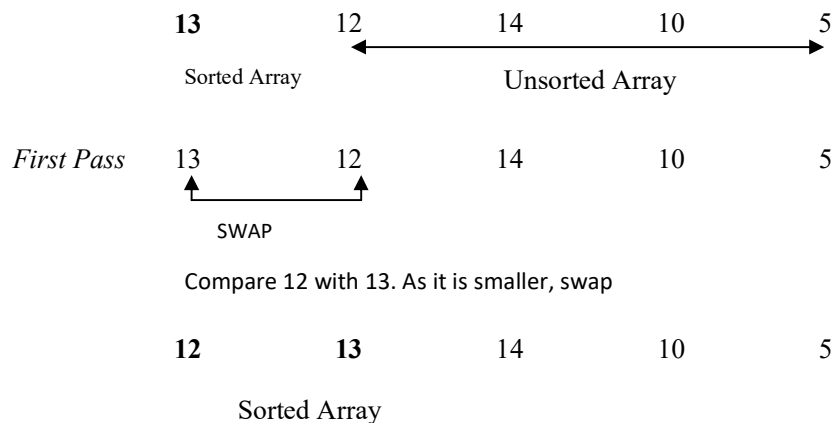
It is a simple sorting algorithm that divide the array into two parts i.e., sorted and unsorted. The algorithm works by iteratively inserting each element of unsorted list into the right place in the sorted list.

Insertion sort is efficient for small data and is a stable and in-place algorithm.

Working of Insertion Sort:

- The first element of the array is assured to be sorted.
- Compare the second element with the element in the sorted array if it is smaller than swap.
- Compare the third element with elements in the sorted array and put it in the correct position.
- Repeat the above steps with other elements until the array is sorted.

Let's understand this with the help of an example:



Second Pass 12 13 14 10 5

Compare 14 with elements in sorted array & swap where required

12 13 14 10 5

Third Pass 12 13 14 10 5

Compare 10 with elements in sorted array & swap where required

12 13 10 14 5

Swap

12 10 13 14 5

Fourth Pass

10 12 13 14 5

10 12 13 14 5

10 12 13 5 14

10 12 5 13 14

10 5 12 13 14

5 10 12 13 14

Pseudocode

def Insertionsort (A):

 for i in range (1, len (A)):

 temp = A[i]

 j = i

 while (j >0 and tmp< A[j-1]):

 A[j] = A[j-1]

 j = j -1

 A[j] = temp

Performance Analysis:

	Time Complexity	Reason
Worst Case	$O(n^2)$	When array is arranged in descending order
Average Case	$O(n^2)$	Irrespective of order of elements
Best Case	$O(n)$	Already sorted array

Space Complexity:

It does not require any auxiliary space i.e., $O(1)$ space complexity. Insertion sort is usually preferred when the array is nearly sorted or when the input size is small.

Check Your Progress-IV

- Given an array of numbers 23, 90, 21, 54, 76. Sort the array using insertion sort.

12.7 Merge Sort

Merge sort is popular sorting algorithm that works on Divide and Conquer strategy. It works by recursively dividing i/p array into two halves, sorting both halves and merging them together to obtain sorted array. It is a stable and not in-place algorithm.

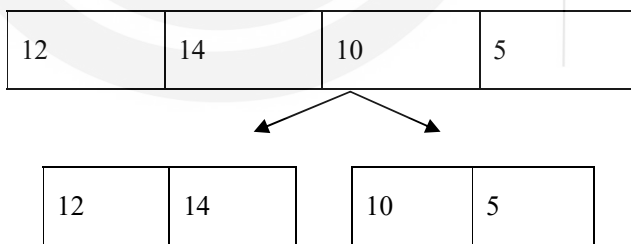
Working of Merge Sort

1. Divide the unsorted array into parts until it cannot be sub-divided- Divide
2. Conquer each sub-array created and sort them – Conquer
Note – If an array contains single element, it is considered sorted.
3. Merge the sorted sub-arrays into correct order.

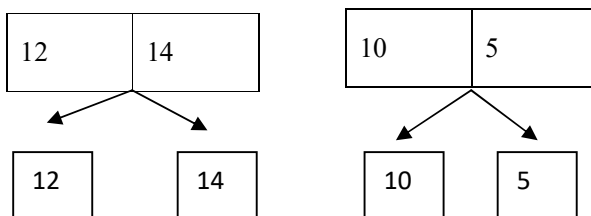
Let's understand it with help of an example:

Given Array: 12 14 10 5

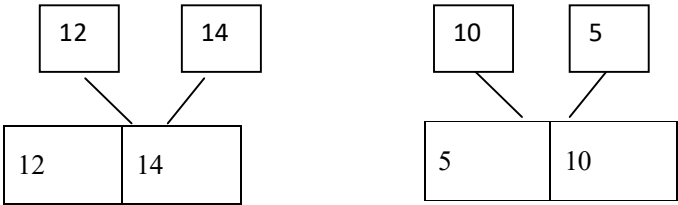
Pass 01: Split the array into equal halves



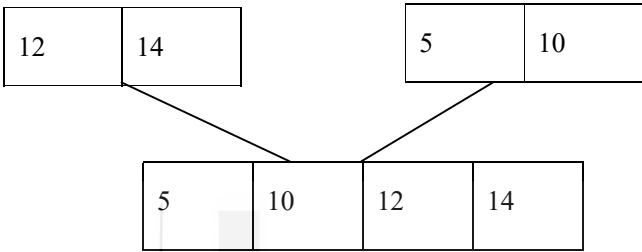
Pass 02: Split the sub-arrays further into equal halves



Pass 03: Sub-arrays cannot be divided further. Therefore, we move to step 02 i.e., conquer, sort, and merge



Pass 04: Conquer, sort , and merge



Pseudocode of Merge Sort:

```
def mergesort (X):
    mid = len (X)/2
    leftpart = X [:mid]
    rightpart = X[mid:]
    mergesort (leftpart)
    mergesort (rightpart)
    while i<len (leftpart) and j <len (rightpart):
        if leftpart[i] <rightpart[j]
            X[k] = leftpart[i]
            i = i + 1
        else
            X[k] = rightpart [j]
            j +=1
        k = k + 1
    while i<len (leftpart):
        X[k] = leftpart [j]
        j = j + 1
        k = k + 1
```

Performance Analysis:

Time Complexity		Reason
Worst Case	O(n log n)	When array is arranged in descending order

Average Case	$O(n \log n)$	Irrespective of order of elements
Best Case	$O(n \log n)$	Already sorted array

Space Complexity:

It is not an in-place sorting algorithm i.e., needs additional memory space of $O(n)$.

Check Your Progress V

- Given an array of numbers 23, 90, 21, 54, 76. Sort the array using bubble sort.

12.8 Quick Sort

Quick sort works on divide and conquer strategy and is also known as partition exchange sort. Just like merge sort, it uses recursive calls to sort the array. The algorithm picks an element as a pivot and partitions the unsorted array around pivot with an aim of placing pivot in the correct position in the sorted array. Further, quick sort is not a stable algorithm i.e., it does not preserve the relative order of same elements. It is in-place algorithm that do not require auxiliary space for sorting.

Working of Quick Sort

- Choose an element as pivot (leftmost element is usually taken as pivot).
- Split the array into 2 parts such that elements smaller than pivot are placed in one part and elements greater than pivot are placed in another part.
- Repeat the same process on both sub-arrays.

Let's understand this with help of an example:

12 14 10 5

Step 01: Considering last element i.e. 5 as pivot

All values are greater than 5 and therefore should be to right of pivot. Swap 5 and 12

5 14 10 12

Step 02: Consider 12 as pivot

Value 10 should be on left of pivot

Value 14 should be on right of pivot

Swap 10 and 14

5 10 14 12

Again, swap 12 and 14

5 10 12 14

Pseudocode of Quicksort

def Partition (X, low, high):

pivot = low

```

swap ( X, pivot, high)
for i in range ( low, high)
    if X[i] <= X[high]
        swap (X, I, low)
        low +=1

swap (X, low, high)
def Quicksort (X, low, high)
    if low < high:
        pivot = Partition (X, low, high)
        Quicksort (X, low, pivot -1 )
        Quicksort (X, pivot +1, high)

```

Performance Analysis

	Time Complexity	Reason
Worst Case	$O(n^2)$	When pivot is the largest or smallest value
Average Case	$O(n \log n)$	When partitions are balanced as much as possible
Best Case	$O(n \log n)$	When pivot divides array roughly into two equal halves

Space Complexity

It is an in-place algorithm, i.e., does not take any auxiliary space.

12.9 Comparing Various Sorting Algorithms

In this section, we will discuss best, average, and worst case time complexity of various sorting algorithms.

Refer to Table 1 for all information about space and time complexity of algorithms.

Table 2 Comparative Analysis of Various Sorting Algorithms

Algorithm	Worst Case	Average Case	Best Case	Stable	In-place
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Yes
Insertion Sort	$O(n^2)$	$O(n^2)$	$O(n)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Yes	No
Quick Sort	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	No	Yes

12.10 Python's Built-In Sorting Functions

There are two main built-in functions in Python for sorting list i.e. `list.sort()` and `sorted`.

Let's understand this with help of an example:

1. `List.sort()`

Given a list of numbers, it sorts the list in ascending or descending order without using any additional space. It makes changes in the original list and does not return a new list.

For instance-

```
l1 = [2, 3, 1, 4]
```

```
l1.sort()
```

It sorts the given list into ascending order

```
l1.sort(reverse = True)
```

It sorts list into descending order.

2. `Sorted()`

This method returns the new sorted list keeping the original list unchanged.

```
L1 = [2, 4, 1, 5]
```

```
L2 = sorted (L1)
```

```
L3 = sorted (L1, reverse = True)
```

Using `sorted()`, we can sort list, tuple or string.

12.11 Summary

- Sorting is the method of arranging elements in either ascending or descending order.
- The algorithm that preserves the relative order of the same elements during sorting is known as a stable sorting algorithm.
- The algorithm that do not need extra space for sorting is known as in-place sorting algorithm.
- Bubble sort works by comparing each pair of elements and swapping elements if they are in the wrong order.
- Selection sort works by repeatedly selecting the smallest element from the unsorted array and swapping it with the first unsorted element.
- Insertion sort is a simple sorting algorithm that divides the array into two parts.
- Merge sort is a popular sorting algorithm that works on Divide and Conquer strategy.

- Quick sort works on divide and conquer strategy and also known as partition exchange sort.

12.12 Solutions/ Answers to Check Your Progress

Check Your Progress I

1.

Internal Sorting	External Sorting
Sorting is done in the main memory (RAM)	Sorting is performed using external memory such as hard disk or secondary storage
Generally faster due to high memory access speed.	Generally slower due to frequent disk I/O operations.
Examples: Quick Sort, Merge Sort, Heap Sort, Bubble Sort	Examples: External Merge Sort, Polyphase Merge Sort

2. A stable sorting algorithm is a method that maintains the relative order of items with equal values.

Example: If a student has equal marks in a subject A and subject B & marks appear in the list as A before B, after stable sorting marks will still be written as A before B.

Bubble Sort, Merge Sort, and Insertion Sort are examples of stable algorithms.

Quick Sort, Heap Sort, and Selection Sort are examples of unstable algorithms.

3. An in-place algorithm is a sorting algorithm that do not require extra amount of memory for sorting elements i.e., can perform sorting in constant time.

It rearranges elements within the original structure and does not use additional data structures like extra arrays or lists; instead,

Quick Sort, Selection Sort, Bubble Sort, and Heap Sort are examples of in-place algorithms.

Check Your Progress-II

- a. Given an array: 23, 90, 21, 54, 76

First Pass:

i=0	23	90	21	54	76
i=1	23	90	21	54	76
i=2	23	21	90	54	76
i=3	23	21	54	90	76

Second Pass:

Third Pass:

Fourth Pass:

b. 67, 21, 12, 10, 9, 7, 5

First Pass:

85

21	12	10	9	7	67	5
21	12	10	7	9	5	67

↓

Largest Element in Last (Sorted Array)

Second Pass:

i=0	21	12	10	7	9	5	67
i=1	12	21	10	7	9	5	67
i=2	12	10	21	7	9	5	67
	12	10	7	21	9	5	67
	12	10	7	9	21	5	67
	12	10	7	9	5	21	67

↓

Sorted Array

Third Pass:

i=0	12	10	7	9	5	21	67
i=1	10	12	7	9	5	21	67
i=2	10	7	12	9	5	21	67
i=3	10	7	9	12	5	21	67
i=4	10	7	9	5	12	21	67

Sorted Array

Fourth Pass:

i=0	10	7	9	5	12	21	67
i=1	7	10	9	5	12	21	67
i=2	7	9	10	5	12	21	67
	7	9	5	10	12	21	67

Sorted Array

Fifth Pass:

i=0	7	9	5	10	12	21	67
	}						
i=1	7	9	5	10	12	21	67
		}					
i=2	7	5	9	10	12	21	67
			}				

Sixth Pass:

i=0	7	5	9	10	12	21	67
	}						
	5	7	9	10	12	21	67

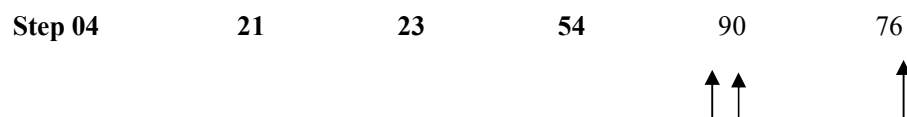
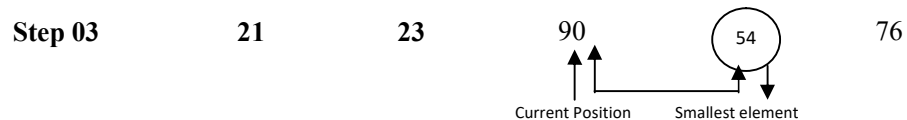
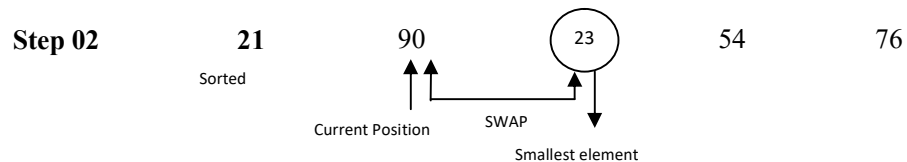
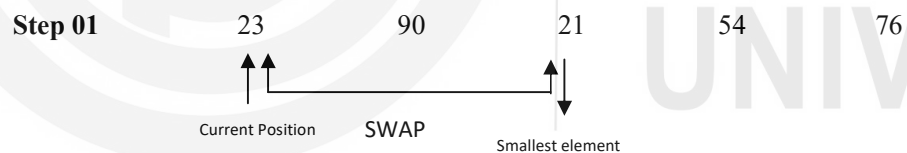
Sorted Array

5 7 9 10 12 21 67

- c. Bubble sort is a stable algorithm as it do not change the position of elements having the same value.

Check Your Progress III

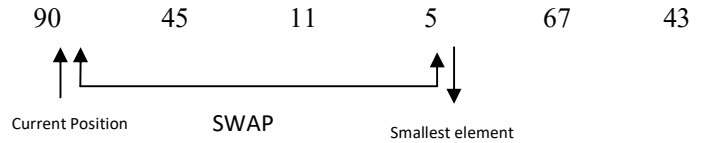
- a. Given 23, 90, 21, 54, 76



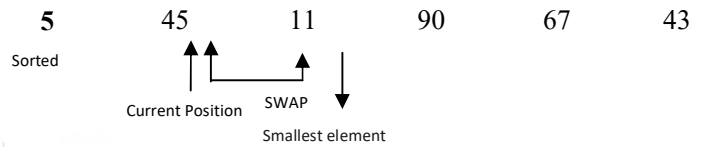
Sorted Array 21 23 54 76 90

b. Given array 90, 45, 11, 5, 67, 43

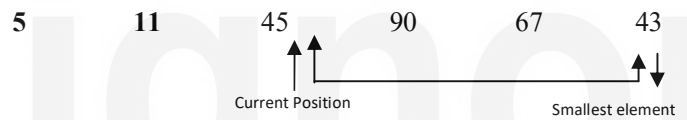
Step 01



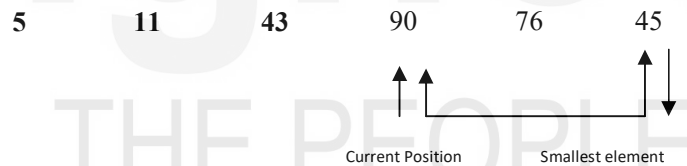
Step 02



Step 03



Step 04



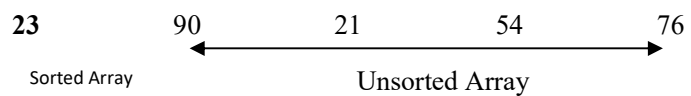
Step 05



Sorted Array 5 11 43 45 76 90

Check Your Progress IV

a. Given the array: 23, 90, 21, 54, 76



First Pass 23 90 21 54 76

 ↑ ↑

 Compare 23 with 90. As 90 is greater, no swap

23 90 21 54 76

Sorted Array

Second Pass 23 90 21 54 76

 ⏟

 Compare 21 with elements in sorted array & swap where required

23 21 90 54 76

⏟

21 23 90 54 76

Third Pass 21 23 90 54 76

 ⏟

 Compare 54 with elements in sorted array & swap where required

21 23 54 90 76

⏟

No Swap

21 23 54 90 76

Fourth Pass 21 23 54 90 76

 ⏟

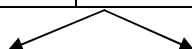
Sorted Array 5 10 12 13 14

Check Your Progress V

Given the array 90, 23, 54, 21

Pass 01: Split the array into equal halves

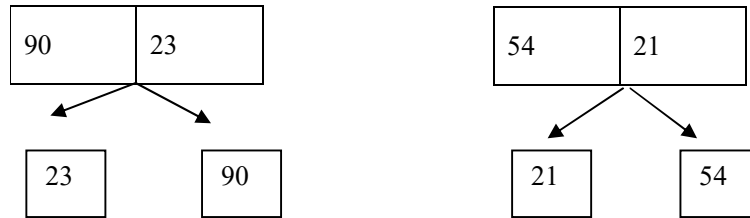
90	23	54	21
----	----	----	----



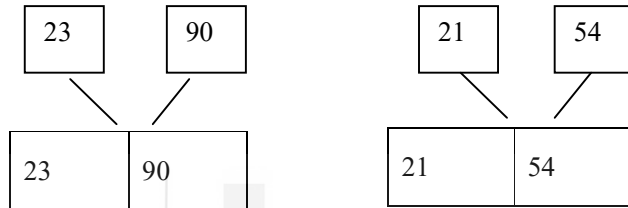
90	23
----	----

54	21
----	----

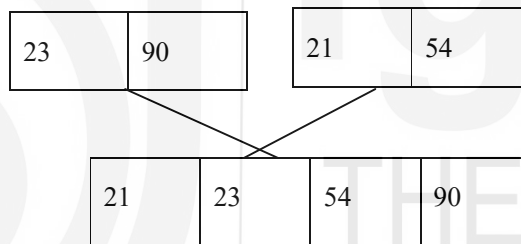
Pass 02: Split the sub-arrays further into equal halves



Pass 03: Sub-arrays cannot be divided further. Therefore, we move to step 02 i.e., conquer, sort, and merge



Pass 04: Conquer, sort, and merge



12.13 Further Readings

Reference Books

1. *Data Structure and Algorithmic Thinking with Python* by Narasimha Karumanchi, CareerMonk Publications, 2016.
2. *Data Structures & Algorithms in Python* by John Canning, Alan Broder, Robert Lafore, Addison-Wesley Professional, 1st edition, 14 October 2022.

Reference Websites

1. https://onlinecourses.nptel.ac.in/noc25_cs59/preview
2. https://www.udemy.com/course/data-structures-algorithms-in-python/?srsltid=AfmBOooSCmtk3ZRIIhQowVXciAk_Ta0HrHO9EPPzVSuM9WR8lvUqH2hZ&couponCode=CP2511241BG1